

DAY 3 · FastAPI learning notes

Pydantic Models for Query Parameters & Request Bodies

Query parameter models · Field validation · Literal · nested body models · HttpUrl · sets

Learning objectives

After working through these notes you should be able to:

- ✓ Group several query parameters into one Pydantic model and read it with `Annotated[Model, Query()]`.
- ✓ Use `Field()` to give a model field a default value, validation rules and docs metadata.
- ✓ Restrict a value to a fixed list of choices with `Literal`.
- ✓ Reject unknown query parameters with `model_config = {"extra": "forbid"}`.
- ✓ Build a request body that contains another model (a nested model) and validate URLs with `HttpUrl`.

Note

Day 3 has no separate summary in the repository, so these notes are built entirely from `day3/main.py`. Every example below is taken from, or run against, that file.

1. Why put parameters in a model?

On Day 2 every query parameter was its own function argument. That is fine for one or two parameters, but a list endpoint quickly needs `limit`, `offset`, `order_by`, `tags`, ... and several endpoints often need the same set. A Pydantic model lets you describe the group **once**, validate it in one place, and reuse it.

2. The query parameter model: FilterParams

day3/main.py

```
class FilterParams(BaseModel):
    model_config = {"extra": "forbid"}

    limit: int = Field(100, gt=0, le=100, description="limite must be between 0-100")
    offset: int = Field(0, ge=0, title="offset")
    order_by: Literal["created_at", "updated_at"] = "created_at"
    tags: list[str] = []
```

Line	What it does
<code>model_config = {"extra": "forbid"}</code>	Any query parameter that is not declared in the model is an error (422).
<code>Field(100, gt=0, le=100, ...)</code>	Default 100; must be 1-100. <code>description</code> appears in <code>/docs</code> .
<code>Field(0, ge=0, title=...)</code>	Default 0; must be 0 or more.
<code>Literal["created_at", "updated_at"]</code>	Only these two exact strings are accepted.
<code>tags: list[str] = []</code>	The parameter may be repeated: <code>?tags=a&tags=b</code> → <code>["a", "b"]</code> .

Field() is to model fields what Query() and Path() are to parameters

The first argument of `Field()` is the default value. After that come the same kinds of rules you used on Day 2 (`gt`, `ge`, `le`, ...) plus metadata like `title` and `description`. Your own comment says it well: “u can use `query(...)` instead” – the rules are the same idea, applied to a model field.

Reading the model from the query string

```
@app.get("/items/")
async def read_items(filter_query: Annotated[FilterParams, Query()]):
    return filter_query
```

A Pydantic model parameter is normally treated as a **JSON request body**. Wrapping it in `Annotated[FilterParams, Query()]` tells FastAPI: build this model, but take its fields from the query string. The function receives a ready, validated `FilterParams` object.

Request (verified)	Result
<code>/items/?limit=5&tags=a&tags=b</code>	200 <code>{"limit": 5, "offset": 0, "order_by": "created_at", "tags": ["a", "b"]}</code>
<code>/items/?limit=0</code>	422 – <code>greater_than</code> (0 is not > 0)
<code>/items/?color=red</code>	422 – <code>extra_forbidden</code> : "Extra inputs are not permitted"
<code>/items/?order_by=name</code>	422 – not one of the Literal values

In your code

The description says “between 0-100”, but `gt=0` means 0 itself is **rejected**. The real range is 1-100. Descriptions are not checked, so keep them in sync with the rules.

3. Nested models in a request body

day3/main.py

```
class Image(BaseModel):
    url: HttpUrl
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

- 1 `Image` is a small model of its own. `HttpUrl` only accepts a valid http/https URL – “bad” gives 422.
- 2 `Item.image` has the type `Image | None`: a model used as a field type is a **nested model**. The JSON body may contain an `"image": {...}` object, validated with `Image`'s rules.
- 3 Fields without a default (`name`, `price`) are required; fields with a default are optional.
- 4 In `update_item`, `item_id` appears in the path → path parameter. `item` has a Pydantic model type → request body.

Verified request and response

```
PUT /items/1
{"name": "Pen", "price": 2.5, "tags": ["office", "office", "blue"]}

-> {"item_id": 1, "item": {"name": "Pen", "description": null, "price": 2.5,
    "tax": null, "tags": ["blue", "office"], "image": null}}
```

Notice that the duplicate **"office"** disappeared: **set[str]** keeps each value once. Sets also have no fixed order, so the order in the response may differ from the request.

A bad nested value is reported with its full location, e.g. **loc**: **["body", "image", "url"]**.

4. How FastAPI decides where a parameter comes from

Parameter looks like...	FastAPI reads it from
name appears in the route path {...}	the path
simple type (int , str , list[str] , ...) not in the path	the query string
a Pydantic model	the JSON request body
a Pydantic model wrapped in Annotated[Model, Query()]	the query string

5. Easily confused

	list[str]	set[str]
Duplicates	kept	removed automatically
Order	kept	not guaranteed
Used in your code for	FilterParams.tags (query)	Item.tags (body)

6. Common mistakes

- Forgetting **Query()** around a model parameter – FastAPI then expects it in the request body.
- Writing a description that disagrees with the rule (**"0-100"** vs **gt=0**).
- Assuming a set keeps the order you sent.
- Expecting unknown parameters to be ignored when **extra** is **forbid** – they cause 422.

7. Revision summary

- A Pydantic model can group query parameters; read it with **Annotated[Model, Query()]**.
- **Field(default, rules..., description=...)** validates and documents a model field.
- **Literal[...]** = fixed choices; **"extra": "forbid"** = unknown parameters are rejected.
- A model used as a field type is a nested model; errors point to the exact nested field.
- **HttpUrl** validates URLs; **set** removes duplicates, **list** keeps them.